

ADRL progress review

Working prototype, still in Wave 3

We have built a shared product foundation, observed a real Claude Code task, and tested controlled job execution and recovery. The next milestone is one trustworthy record of exactly what a completed task produced.

What ADRL is for

Think of ADRL as a shared supervisor around AI coding tools. The tool works on files. The model suggests the next action. ADRL aims to apply rules, record activity and check the evidence. One shared core should support multiple coding tools.

What we have achieved

Shared services	Working local connections, identity checks and activity history provide a common foundation for future integrations.
Real observation	All 18 tool actions in the checked Claude Code session trace matched ADRL records, including a failed action.
Separate verification	The first repair passed the old tests but failed compatibility checks. The corrected repair passed all eight checks.
Protected evidence	Encrypted captures preserve selected work after later edits. Task records distinguish started and interrupted work.
Controlled recovery	Fixed test jobs passed normal stop, lost confirmation, access-revocation and owner-process-death scenarios.

The latest evidence

895 tests and all 11 engineering checks passed. The latest controlled experiment received the original identities for all 13 test jobs and confirmed their removal. It used synthetic jobs, with no real task payloads or model calls.

These results support the tested behaviors. They do not measure percentage completion, general reliability or business value.

The path to a useful product

Stronger evidence, with clear limits

The current product remains a local prototype. Claude Code has one limited real observation pilot. OpenCode and Codex integration are planned. Broad workflow reliability, production readiness, lower cost and better coding quality still need proof.

Testing is already changing the design

We added a controlled execution boundary after a helper kept writing beyond its parent process. We strengthened future-access denial after finding a stale-key recovery weakness. We separated preparation and supervision timeouts after reproducing a lost-response mechanism. The earlier real creation failure still has an unknown cause.

We can test proposed improvements

A proposed checker caught three defects the earlier checker missed: **7 of 7** correct classifications versus **4 of 7**. These were visible variants of one task, authored with the candidate. Fresh, independently reviewed examples must test whether the benefit transfers.

The next milestones

Now: Wave 3	Account for temporary decrypted copies and preserve the exact output at task completion.
Waves 4-5	Check fresh tasks and repeat the workflow through OpenCode to prove the shared core transfers.
Waves 6-8	Test real controls, measure quality and cost, and qualify a supported local product.
Later waves	Use approved evidence to learn and propose changes, with independent tests and approval.

Maturity and improvement: the latest slice updated eight decision records and preserved all 77 formal grades. Today we record failures, test changes and update the ledger. Recursive self-improvement would also improve how future changes are found and evaluated. That capability remains unproven.

Leadership: no immediate input blocks the next offline work. Broader pilots will need reviewer assignments, fresh task examples and explicit access, retention and spending choices.

Sources: ADRL live observation pilot and measured improvement experiment, 7 September 2026. ADRL transport/receipt report, execution state and implementation journey, 8 September 2026. The slide deck carries detailed source references in speaker notes.